

Descrição do Projeto Semestral de POO (25/08/2026)

João Pedro de Jesus Cândido Silva

R.A. 23.01416-4

1. Título do projeto

Nexo Invest — Simulador de Investimentos e Planejamento Financeiro

2. Descrição do problema ou ideia

O projeto consiste no desenvolvimento de uma aplicação desktop voltada ao acompanhamento, análise e simulação de investimentos.

A aplicação permitirá que o usuário consulte informações atualizadas de diferentes ativos financeiros por meio de serviços externos, crie carteiras de investimento simuladas e acompanhe sua evolução através de indicadores, cálculos e visualizações gráficas.

O sistema pretende centralizar em uma única interface informações que normalmente se encontram distribuídas entre diferentes sites e ferramentas, possibilitando que o usuário registre operações simuladas, acompanhe sua carteira, analise ativos e utilize métodos de apoio à tomada de decisão.

Além das funcionalidades principais, poderão ser desenvolvidos recursos complementares relacionados a alertas de preço, planejamento financeiro pessoal e inteligência artificial, de acordo com o andamento do desenvolvimento.

O sistema terá caráter educacional e de simulação, não realizando operações financeiras reais.



3. Objetivo principal

Desenvolver, utilizando Python e Programação Orientada a Objetos, uma aplicação gráfica capaz de integrar dados financeiros obtidos através de serviços externos com informações persistidas localmente, permitindo a criação e o acompanhamento de carteiras simuladas, análise de ativos e visualização de indicadores financeiros de maneira clara e interativa.

O projeto também busca aplicar de maneira prática conceitos de arquitetura de software, encapsulamento, composição, associação, herança e polimorfismo, além da integração entre interface gráfica, banco de dados, serviços externos e lógica de negócio.

4. Principais funcionalidades previstas

As funcionalidades principais previstas são:

- consulta de cotações e informações de ativos financeiros através de API ou serviço externo;
- pesquisa de ativos por código ou nome;
- criação, edição e exclusão de carteiras de investimento simuladas;
- registro de operações simuladas de compra e venda;
- acompanhamento das posições presentes em cada carteira;
- cálculo do valor investido, valor atual, preço médio, lucro ou prejuízo e rentabilidade;
- visualização da distribuição da carteira entre diferentes ativos;
- apresentação de gráficos e indicadores financeiros;
- realização de cálculos de apoio à análise de investimentos, incluindo estimativas de preço-teto e outros indicadores;
- criação de uma área específica para análise detalhada de ativos;
- armazenamento persistente das carteiras, operações e configurações do usuário;
- validação de dados inseridos e tratamento de situações de erro, como indisponibilidade do serviço externo ou código de ativo inexistente;
- possibilidade de criação de alertas configuráveis relacionados ao preço dos ativos.

Como funcionalidades adicionais, caso o cronograma de desenvolvimento permita, pretende-se implementar:

- envio de alertas através de e-mail ou serviço de mensagens;
- módulo de planejamento financeiro pessoal, relacionando receitas, despesas, objetivos e investimentos;

- integração com um modelo de linguagem para geração de explicações e análises auxiliares relacionadas ao portfólio e à educação financeira.

5. Tecnologias e bibliotecas que pretende utilizar

A aplicação será desenvolvida principalmente utilizando:

Python 3.x como linguagem principal;

PySide6 para construção da interface gráfica;

SQLite para armazenamento local das carteiras, transações, alertas e demais informações do usuário;

Requests, HTTPX ou biblioteca equivalente para comunicação com serviços externos;

API ou serviço de informações do mercado financeiro para obtenção das cotações e informações dos ativos;

Matplotlib, PyQtGraph ou biblioteca equivalente para criação de gráficos e visualizações financeiras;

além de bibliotecas auxiliares do ecossistema Python para processamento, validação e manipulação dos dados.

Para funcionalidades adicionais poderão ser utilizadas APIs de comunicação, bibliotecas de e-mail e APIs de modelos de linguagem.

6. Utilização de Programação Orientada a Objetos

A Programação Orientada a Objetos será utilizada como base da arquitetura da aplicação, buscando representar de maneira coerente os elementos do domínio financeiro e distribuir adequadamente as responsabilidades do sistema entre diferentes objetos.

A utilização dos conceitos de orientação a objetos não será realizada apenas com a finalidade de demonstrar recursos da linguagem. As relações entre as classes deverão refletir necessidades reais do domínio e da arquitetura da aplicação, evitando a criação artificial de hierarquias ou relacionamentos que não contribuam para o funcionamento do sistema.

Entre as principais classes inicialmente previstas encontram-se:

Ativo, representando um ativo financeiro identificado principalmente por seu código ou símbolo, além de outras informações relevantes obtidas a partir dos serviços de dados de mercado;

Carteira, responsável por representar o conjunto de investimentos simulados pelo usuário e coordenar suas posições;

Posição, representando a quantidade mantida de determinado ativo e informações calculadas a partir das operações realizadas, como quantidade acumulada, preço médio, valor atual e resultado;

Transação, responsável pelo registro das operações simuladas de compra e venda;

AlertaPreco, representando uma condição configurada pelo usuário para acompanhamento do preço de determinado ativo.

Além dessas classes pertencentes diretamente ao domínio financeiro, a aplicação contará com classes responsáveis por:

- obtenção e tratamento de dados de mercado;
- execução de cálculos e análises financeiras;
- registro e recuperação das informações armazenadas;
- implementação dos casos de uso da aplicação;
- comunicação entre a lógica do sistema e a interface gráfica.

Encapsulamento

O encapsulamento será utilizado para manter os dados e comportamentos relacionados a cada conceito dentro do objeto responsável por eles.

Dessa forma, as regras do domínio não ficarão distribuídas diretamente pela interface gráfica ou por diferentes partes do programa.

Por exemplo, uma transação poderá ser responsável por validar seus próprios dados, impedindo a criação de operações com quantidade ou preço inválidos. De maneira semelhante, a carteira poderá disponibilizar operações para consulta de patrimônio, posições existentes e resultados consolidados sem expor desnecessariamente sua representação interna.

Essa abordagem permite que as regras de negócio permaneçam centralizadas e reduz o acoplamento entre as diferentes partes da aplicação.

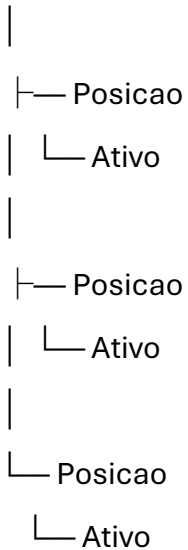
Composição

A composição será o principal mecanismo utilizado para representar os relacionamentos entre os objetos do domínio.

Uma **Carteira** será formada a partir de diferentes **Posições**, cada uma representando a situação consolidada de determinado ativo dentro da carteira.

De forma conceitual:

Carteira

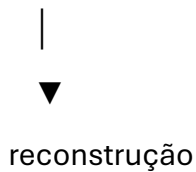


As posições não serão tratadas como registros independentes da carteira, mas como partes que compõem sua representação em determinado momento.

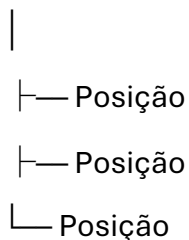
Além disso, o estado da carteira poderá ser reconstruído a partir do conjunto de transações registradas pelo usuário.

O fluxo conceitual será semelhante a:

Transações



Carteira



Cada **Posição** conterá as informações necessárias sobre o **Ativo** correspondente e os valores calculados a partir das operações realizadas, como quantidade, preço médio e valor atual.

A composição também poderá ser empregada em outras partes da aplicação sempre que um objeto for constituído por outros objetos necessários para representar seu estado ou executar sua responsabilidade.

Herança

A utilização de herança não será definida antecipadamente apenas para demonstrar o conceito.

Inicialmente, todos os ativos financeiros poderão ser representados pela classe **Ativo**, diferenciados por seus próprios dados e características, sem a necessidade de criar classes como **Acao**, **FII** e **ETF** exclusivamente para estabelecer uma hierarquia.

Uma estrutura como:

Ativo

└─ Acao

└─ FII

└─ ETF

somente seria adequada caso essas classes apresentassem diferenças reais de estado ou comportamento que justificassem a relação de especialização.

Como a aplicação, em sua versão inicial, utiliza os ativos principalmente como elementos identificados por um símbolo e vinculados a cotações, transações e posições, a criação dessa hierarquia adicionaria complexidade sem uma necessidade concreta do domínio.

Caso, durante a evolução do projeto, sejam identificadas situações em que uma classe especializada realmente represente uma relação do tipo “**é um**” e possua comportamento próprio relevante, a utilização de herança poderá ser reavaliada.

Polimorfismo

O polimorfismo também não será introduzido artificialmente por meio de uma hierarquia de ativos.

Entretanto, esse conceito poderá surgir de forma natural em componentes da aplicação que possuam diferentes implementações capazes de executar a mesma responsabilidade.

Um exemplo possível são os serviços responsáveis pela obtenção de dados de mercado.

A aplicação poderá trabalhar com um contrato conceitual semelhante a:

obter_cotacao(simbolo)

Diferentes componentes poderão oferecer essa mesma operação utilizando fontes distintas de dados.

Por exemplo:

YahooFinanceProvider

└─ obter_cotacao()

OutroMarketDataProvider

└─ obter_cotacao()

Dessa maneira, o restante da aplicação poderá utilizar um provedor de dados sem depender diretamente dos detalhes de funcionamento da API utilizada.

Em Python, esse comportamento poderá ser obtido por meio de objetos que respeitem uma mesma interface de utilização, permitindo substituir uma implementação por outra sem alterar a lógica responsável pelo consumo desses dados.

Caso seja necessário formalizar esse contrato durante o desenvolvimento, poderá ser utilizada uma abstração apropriada, desde que exista uma necessidade arquitetural concreta para isso.

Organização dos principais objetos

De maneira simplificada, os relacionamentos do domínio poderão ser representados como:

Carteira

|

└─ composta por Posições

| |

| └─ cada Posição representa um Ativo

|

└─ construída a partir das Transações

Transação

|

└─ registra uma operação envolvendo um Ativo

AlertaPreco

|

└─ mantém as informações necessárias

para acompanhar determinado Ativo

A prioridade será, portanto, representar os relacionamentos por meio de composição sempre que os objetos fizerem parte da estrutura ou do estado de outro objeto, evitando a introdução de associações conceituais desnecessárias.

Com essa estrutura, a aplicação demonstrará principalmente a utilização de **classes e objetos, encapsulamento, abstração e composição**, mantendo a possibilidade de utilização de **polimorfismo** quando houver componentes realmente intercambiáveis.

A **herança** somente será utilizada caso o desenvolvimento revele uma relação de especialização concreta e justificável, evitando sua aplicação apenas para satisfazer formalmente um conceito de Programação Orientada a Objetos.

Por fim, a interface gráfica será mantida separada da lógica de negócio e da camada de persistência. Essa separação permitirá que as classes do domínio permaneçam independentes da tecnologia utilizada na interface e no banco de dados, favorecendo modularização, manutenção, testes e evolução futura do sistema.

A interface gráfica será separada da lógica de negócio e da camada de persistência, permitindo maior modularização, manutenção e reaproveitamento do código.

7. Integrações implementadas

O projeto possuirá inicialmente duas formas principais de integração.

A primeira será a integração com um **banco de dados SQLite**, utilizado para armazenar, consultar, modificar e remover informações relacionadas às carteiras simuladas, operações, configurações e alertas.

A segunda será a integração com um **serviço externo de informações financeiras**, através do qual a aplicação obterá cotações e outras informações dos ativos.

Como integração adicional, pretende-se avaliar a utilização de um serviço de e-mail ou mensagens para envio automático de alertas de preço.

Também poderá ser adicionada posteriormente uma integração com serviços de Inteligência Artificial.

8. Elementos gráficos, multimídia e organização da interface

A aplicação utilizará diferentes elementos gráficos com função direta na interpretação dos dados financeiros.

Serão utilizados dashboards e gráficos para apresentar informações como:

- evolução do patrimônio;
- composição da carteira;
- distribuição dos investimentos;
- rentabilidade;
- lucro ou prejuízo;
- evolução histórica da cotação dos ativos;
- comparação entre valor atual e valores calculados de referência.

Também serão utilizados elementos visuais interativos na interface, como cartões de indicadores, tabelas dinâmicas, ícones, filtros, seletores, barras de progresso e indicadores visuais de variação positiva ou negativa.

A identidade visual será desenvolvida especificamente para o projeto, buscando oferecer uma interface moderna, organizada, intuitiva e coerente entre as diferentes áreas da aplicação.

8.1. Estrutura geral da interface

A interface gráfica será desenvolvida utilizando **PySide6**, buscando oferecer uma experiência semelhante à de aplicações financeiras modernas, com navegação simples, identidade visual própria e apresentação clara das informações.

A aplicação será estruturada inicialmente a partir de uma **janela principal**, contendo uma área de navegação persistente e uma região central responsável pela exibição das diferentes páginas do sistema.

A organização geral prevista será semelhante à seguinte:



A navegação entre as páginas poderá ser implementada utilizando componentes como `QStackedWidget`, permitindo manter uma única janela principal e alterar dinamicamente seu conteúdo de acordo com a opção selecionada pelo usuário.

8.2. Dashboard

Será a página inicial da aplicação e apresentará uma visão resumida da situação das carteiras do usuário.

Poderá apresentar:

- patrimônio total;
- valor total investido;
- lucro ou prejuízo acumulado;
- rentabilidade;
- variação diária;
- distribuição da carteira;
- evolução patrimonial;
- resumo das principais posições;
- alertas recentes;
- ativos com maiores valorizações ou desvalorizações.

Essa tela utilizará cartões de indicadores e gráficos para facilitar a interpretação das informações.

8.3. Mercado

Permitirá consultar ativos financeiros disponíveis através do serviço externo integrado à aplicação.

Entre os recursos previstos estão:

- pesquisa por código ou nome;
- cotação atual;
- variação diária;
- informações básicas do ativo;
- histórico de preços;
- gráfico de evolução;
- acesso rápido à página de análise do ativo;
- possibilidade de adicionar o ativo a uma carteira ou criar um alerta.

8.4. Carteiras

Permitirá visualizar e administrar as carteiras simuladas cadastradas pelo usuário.

Nessa área será possível:

- criar uma nova carteira;
- editar uma carteira existente;
- excluir uma carteira;
- visualizar patrimônio e rentabilidade;
- acessar os detalhes de cada carteira.

8.5. Detalhes da carteira

Apresentará as informações específicas de uma carteira selecionada.

Poderá conter:

- lista das posições;
- quantidade de cada ativo;
- preço médio;
- preço atual;
- valor investido;
- valor atual;
- lucro ou prejuízo;
- rentabilidade por ativo;
- distribuição percentual da carteira;
- histórico de operações;

- gráficos relacionados à composição e evolução da carteira.

Também permitirá registrar novas operações simuladas de compra e venda.

8.6. Análise de ativos

Será destinada à realização de cálculos e apresentação de indicadores utilizados como apoio à análise de investimentos.

Poderá incluir:

- preço atual;
- histórico de preços;
- indicadores fundamentalistas disponíveis;
- informações sobre dividendos e proventos;
- estimativas de preço-teto;
- margem entre preço atual e valor calculado;
- diferentes métodos de análise implementados no sistema.

Essa estrutura permitirá futuramente acrescentar novos métodos de avaliação sem modificar significativamente as demais partes da aplicação.

8.7. Alertas

Permitirá criar e administrar condições relacionadas à cotação dos ativos.

O usuário poderá definir, por exemplo:

PETR4

Preço atual: R\$ 32,40

Avisar quando:

Preço $\leq$ R\$ 30,00

ou:

WEGE3

Preço atual: R\$ 42,50

Avisar quando:

Preço $\geq$ R\$ 50,00

Os alertas poderão inicialmente ser exibidos dentro da própria aplicação e, posteriormente, poderão ser associados ao envio de notificações por e-mail ou outro serviço externo.

8.8. Planejamento financeiro

Caso seja implementada dentro do período disponível, essa área permitirá relacionar investimentos com a situação financeira pessoal simulada pelo usuário.

Poderá conter:

- salário ou renda principal;
- rendas adicionais;
- despesas fixas;
- despesas variáveis;
- capacidade mensal de investimento;
- reserva financeira;
- objetivos;
- prazo dos objetivos;
- acompanhamento da evolução financeira.

Essa funcionalidade será considerada complementar ao núcleo principal do projeto.

8.9. Assistente baseado em Inteligência Artificial

Como funcionalidade adicional, poderá existir uma página destinada à interação com um modelo de linguagem.

O assistente poderá utilizar informações disponibilizadas pela aplicação para fornecer:

- explicações sobre indicadores;
- informações educacionais;
- auxílio na interpretação da carteira;
- análises descritivas;
- explicações sobre diversificação e planejamento financeiro.

Essa funcionalidade terá caráter auxiliar e educacional e não será apresentada como mecanismo automático de recomendação financeira.

8.10. Configurações

Permitirá administrar preferências gerais da aplicação, como:

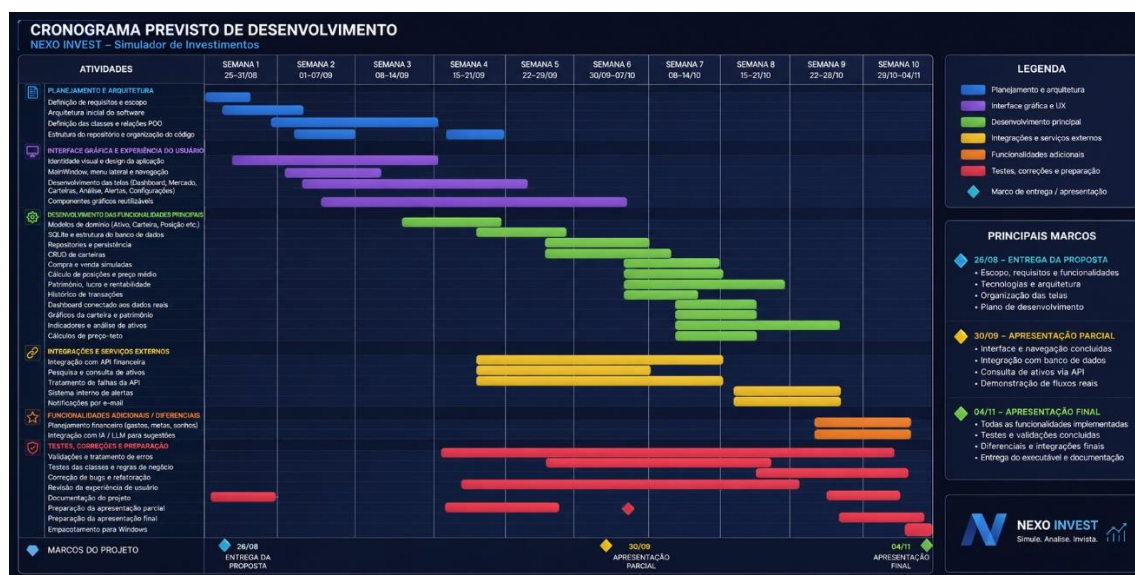
- configurações dos serviços externos;
- preferências de interface;
- parâmetros dos alertas;
- opções relacionadas às notificações;
- configurações das funcionalidades adicionais.

A estrutura proposta permitirá que novas telas e funcionalidades sejam incorporadas durante o desenvolvimento sem exigir alterações significativas na navegação principal.

9. Cronograma previsto de desenvolvimento

O desenvolvimento será realizado de maneira incremental, priorizando inicialmente a arquitetura da aplicação, a interface gráfica e as funcionalidades essenciais.

Como a apresentação parcial ocorrerá em **30/09**, pretende-se chegar a essa etapa com a estrutura visual praticamente completa e com uma primeira versão funcional envolvendo Programação Orientada a Objetos, banco de dados e comunicação com serviços externos.



25/08 a 31/08 — Definição e estrutura inicial

Objetivos:

- finalizar e entregar a proposta do projeto;
- definir requisitos funcionais e não funcionais;
- definir o escopo obrigatório e funcionalidades adicionais;
- escolher a identidade visual;
- definir a arquitetura inicial;
- definir as principais classes;
- criar o repositório Git;
- criar a estrutura inicial de diretórios;
- iniciar a estrutura principal da aplicação PySide6.

Resultado esperado: projeto estruturado e preparado para o início da implementação.

01/09 a 07/09 — Estrutura principal da interface

Objetivos:

- desenvolver a MainWindow;
- desenvolver menu lateral;
- implementar sistema de navegação;
- criar cabeçalho e componentes reutilizáveis;
- definir tema, tipografia, ícones e identidade visual;
- criar estrutura das páginas principais;
- implementar navegação através de QStackedWidget ou mecanismo equivalente.

Resultado esperado: aplicação navegável entre suas principais áreas.

08/09 a 14/09 — Desenvolvimento das telas

Objetivos:

- estruturar Dashboard;
- estruturar página Mercado;
- estruturar página Carteiras;
- estruturar detalhes da carteira;
- estruturar página de Análise;
- estruturar página de Alertas;
- estruturar Configurações;
- criar componentes gráficos reutilizáveis;
- utilizar inicialmente dados simulados quando necessário.

Resultado esperado: interface e fluxo de navegação praticamente completos.

15/09 a 21/09 — Arquitetura POO e banco de dados

Objetivos:

- implementar classes principais do domínio;
- implementar Ativo, Carteira, Posicao, Transacao e AlertaPreco;
- implementar relacionamentos entre objetos;
- estabelecer separação entre interface, regras de negócio e persistência;
- configurar banco SQLite;
- implementar camada de acesso aos dados;
- criar operações CRUD;
- integrar criação, edição e exclusão de carteiras à interface.

Resultado esperado: aplicação utilizando efetivamente Programação Orientada a Objetos e persistindo informações no banco.

22/09 a 29/09 — API financeira e preparação da apresentação parcial

Objetivos:

- integrar o serviço externo de informações financeiras;
- implementar pesquisa de ativos;
- obter cotações reais;
- apresentar informações recebidas na interface;
- tratar falhas de conexão;
- tratar ativos inexistentes;
- integrar pelo menos um fluxo completo entre interface, serviços, objetos e banco de dados;
- revisar arquitetura e principais classes;
- preparar demonstração da versão parcial.

Até essa etapa, pretende-se possuir pelo menos os seguintes fluxos funcionais:

Interface



Criar carteira



Objeto Carteira



Repository



SQLite

e:

Pesquisar ativo



Serviço de mercado



API externa



Objeto Ativo



Interface gráfica

Resultado esperado: versão funcional preparada para a apresentação parcial.

30/09 — Apresentação Parcial de Desenvolvimento

Na apresentação parcial serão demonstrados:

- objetivo e escopo;
- arquitetura inicial;
- principais classes;
- aplicação gráfica em funcionamento;
- navegação entre as páginas;
- persistência utilizando SQLite;
- estágio da integração com a API financeira;
- funcionalidades já implementadas;
- dificuldades encontradas;
- funcionalidades pendentes;
- planejamento até a entrega final.

O objetivo é que a interface e sua lógica de navegação já estejam praticamente concluídas nessa etapa, permitindo concentrar o restante do desenvolvimento principalmente nas regras de negócio e funcionalidades.

01/10 a 07/10 — Sistema completo de carteiras

Objetivos:

- implementar operações simuladas de compra;
- implementar operações simuladas de venda;
- atualizar automaticamente posições;
- calcular preço médio;
- calcular valor investido;
- obter valor atual das posições;
- calcular lucro ou prejuízo;
- calcular rentabilidade;
- implementar histórico de transações;
- integrar completamente essas funcionalidades ao banco.

Resultado esperado: gerenciamento de carteiras funcional.

08/10 a 14/10 — Dashboard, gráficos e análise

Objetivos:

- conectar Dashboard aos dados reais da aplicação;
- implementar gráfico de composição da carteira;
- implementar evolução patrimonial;
- implementar histórico das cotações;
- criar indicadores visuais;

- implementar métodos de análise;
- implementar cálculos de preço-teto;
- melhorar visualização dos dados.

Resultado esperado: núcleo funcional completo e visualmente integrado.

15/10 a 21/10 — Alertas e integrações adicionais

Objetivos:

- criar sistema de alertas de preço;
- armazenar alertas no banco;
- verificar condições configuradas;
- exibir notificações dentro da aplicação;
- implementar, caso viável, envio de alertas por e-mail;
- revisar tratamento de exceções.

Resultado esperado: funcionalidades essenciais finalizadas e primeira integração adicional implementada.

A partir dessa etapa, o núcleo principal do projeto deverá ser considerado estável, evitando a inclusão de alterações que possam comprometer funcionalidades já concluídas.

22/10 a 28/10 — Funcionalidades complementares

Caso todas as funcionalidades principais estejam estáveis, poderão ser desenvolvidos alguns dos diferenciais previstos, como:

- planejamento financeiro;
- cadastro de receitas e despesas;
- objetivos financeiros;
- integração com modelo de linguagem;
- melhorias no sistema de alertas;
- novos indicadores;
- outras melhorias identificadas durante os testes.

Essas funcionalidades serão implementadas somente se não comprometerem a qualidade e estabilidade das funcionalidades principais.

Resultado esperado: ampliação do projeto através de diferenciais tecnicamente relevantes.

29/10 a 02/11 — Testes e finalização

Objetivos:

- testar todos os principais fluxos;
- corrigir erros;
- testar entradas inválidas;
- testar indisponibilidade da API;
- verificar persistência dos dados;
- revisar usabilidade;
- revisar aparência e consistência visual;
- realizar refatoração quando necessária;
- revisar nomenclatura e documentação;
- preparar versão executável para Windows;
- preparar apresentação final.

Resultado esperado: versão final estável.

03/11 — Preparação da apresentação

Objetivos:

- congelar a versão utilizada na demonstração;
- realizar ensaio completo;
- verificar ambiente de execução;
- preparar dados de demonstração;
- preparar slides;
- revisar explicação da arquitetura;
- revisar conceitos de POO utilizados;
- revisar principais classes e métodos;
- preparar explicação das integrações;
- levantar dificuldades, limitações e melhorias futuras.

04/11 — Apresentação e entrega final

Será apresentada a versão final do **Nexo Invest**, demonstrando suas principais funcionalidades, arquitetura orientada a objetos, interface gráfica, banco de dados, comunicação com serviços externos, elementos gráficos e demais funcionalidades concluídas durante o semestre.

10. Resultados esperados

Ao final do projeto, espera-se obter uma aplicação desktop funcional que permita ao usuário acompanhar e simular investimentos através de uma interface gráfica organizada e intuitiva.

O sistema deverá permitir a consulta de informações atualizadas do mercado financeiro, gerenciamento persistente de carteiras simuladas, realização de operações de compra e venda fictícias, cálculo de indicadores e análise visual da composição e desempenho das carteiras.

Espera-se também que a arquitetura desenvolvida demonstre de forma clara a aplicação dos conceitos de Programação Orientada a Objetos estudados durante a disciplina, mantendo separadas as responsabilidades relacionadas à interface gráfica, lógica de negócio, comunicação com serviços externos e armazenamento de dados.

Como resultado adicional, pretende-se desenvolver uma arquitetura extensível que permita incorporar posteriormente novos tipos de ativos, métodos de análise, serviços de obtenção de dados, mecanismos de notificação e recursos de planejamento financeiro ou inteligência artificial sem exigir alterações significativas na estrutura principal da aplicação.